

Contents

1	DD03: 監査ハッシュチェーン (運営者システム)	1
1.1	0. 文書情報	1
1.2	1. 対象範囲	1
1.3	2. 収録ロジック・対応章	1
1.4	3. 詳細設計本文	2
1.4.1	3.1 監査ハッシュチェーン構造	2
1.4.2	3.2 書込トランザクション (D-04)	2
1.4.3	3.3 日次ハッシュチェーン検証 (AuditChainVerifierWorker、D-04)	2
1.4.4	3.4 不一致検出時の通知ポリシー	3
1.4.5	3.5 監査ログ検索 API	3
1.4.6	3.6 監査エクスポート (HMAC 署名、D-17)	4
1.4.7	3.7 3 区分保持 (D-08、付録 F)	4
1.4.8	3.8 accounts_retired 永久保持	4
1.4.9	3.9 構造化ログ (監査関連)	4
1.4.10	3.10 主要エラーコード	5
1.5	4. 関連設計	5
1.6	5. テスト観点	5
1.6.1	5.1 ユニットテスト	5
1.6.2	5.2 結合テスト (Miniflare)	5
1.6.3	5.3 E2E テスト (Playwright)	5
1.6.4	5.4 負荷試験	6
1.6.5	5.5 受入条件マッピング	6
1.7	6. 未確定事項・確認事項	6

1 DD03: 監査ハッシュチェーン (運営者システム)

1.1 0. 文書情報

項目	内容
文書名	DD03: 監査ハッシュチェーン (運営者システム)
詳細設計 ID	DD03
対象システム	FAQ AI ウィジェット SaaS / 運営者システム
関連機能 ID	FR-229, FR-230, FR-231, FR-232, NFR-306, NFR-602, NFR-705, D-04, D-08, D-17
作成日	2026-05-17
版数	v1.0
ステータス	承認済

1.2 1. 対象範囲

種別	ID	名称
機能	FR-229	監査ログ全件記録
機能	FR-230	監査ログ検索
機能	FR-231	監査ログハッシュチェーン
機能	FR-232	監査ログエクスポート (HMAC 署名)
画面	SCR-096	監査ログ閲覧・エクスポート
API	GET /audit-logs	検索
API	POST /audit-logs/exports	エクスポート開始
API	GET /audit-logs/exports/{job_id}	エクスポート進捗
テーブル	audit_logs	ハッシュチェーン主管 (運営者主管)

1.3 2. 収録ロジック・対応章

元章	元タイトル	概要
§ 6 監査ログ機能詳細(参照)	ハッシュチェーン書込・検索・エクスポート	記録 → 検索 → エクスポート
§ 12 ハッシュチェーン詳細(参照)	チェーン構造 + 管理	prev_hash + record_hash + HKDF info=audit-chain
§ 14.2.2	AuditChainVerifierWorker 擬似コード	日次 02:00 JST 全件再計算
§ 16 構造化ログ(監査関連)	監査関連の構造化ログ要件	actor_id / action / trace_id
付録 F	3 区分保持の物理対応	action_code × retention_class 全件 マップ

1.4 3. 詳細設計本文

1.4.1 3.1 監査ハッシュチェーン構造

各 audit_logs レコードは前レコードの record_hash を prev_hash 列に保持し、暗号学的に連結する。

```
record_hash = HMAC-SHA256(
    key = HKDF(masterKey, info="audit-chain"),
    message = prev_hash || canonical_json({
        actor_id, action, target_type, target_id,
        payload_hash, retention_class, occurred_at
    })
)
```

項目	仕様
初期値 prev_hash	"0" * 64(全 0、ジェネシス)
HMAC アルゴリズム	HMAC-SHA256
HKDF info	audit-chain
record_hash 長	64 文字(16 進エンコード)
順序保証	(occurred_at, id) の昇順、ULID 採番で時系列単調保証
改ざん検知	1 レコードを書換えると以降全レコードの record_hash が不一致

1.4.2 3.2 書込トランザクション (D-04)

監査ログ INSERT は次のトランザクションで実行する:

```
BEGIN;
-- 直前レコードを排他取得
SELECT id, record_hash FROM audit_logs
ORDER BY occurred_at DESC, id DESC LIMIT 1
FOR UPDATE;
-- 新レコード INSERT
INSERT INTO audit_logs (
    id, actor_id, action, target_type, target_id,
    payload, payload_hash, retention_class,
    prev_hash, record_hash, occurred_at, trace_id
) VALUES (?, ..., ?, computed_record_hash, ?, ?);
COMMIT;
```

D1 のトランザクション境界制約(現状 batch API ベース)を踏まえ、実装では `db.batch([...])` で SELECT + INSERT を 1 つの論理単位として送出する。並行 INSERT で同一 prev_hash を使う競合は影響行数 = 0 で検出し、リトライ(最大 3 回)+ それでも失敗時は `audit.write.conflict` 監査 + 運営者 high alert。

1.4.3 3.3 日次ハッシュチェーン検証 (AuditChainVerifierWorker、D-04)

```
function auditChainVerify():
    verify_date = today
    runId = ULID()
    audit_logs(audit.chain.verify.start, 5y, payload={verify_date, run_id: runId})

    prev_hash = "0" * 64
    mismatch_count = 0
```

```

cursor = null
do:
    rows = D1.query("SELECT id, prev_hash, record_hash, ... FROM audit_logs WHERE id > ? OR
for r in rows:
    expected = sha256(prev_hash + canonical_json(r.{actor_id, action, ..., retention_cl
    if expected != r.record_hash:
        audit_logs(audit.chain.verify.fail, 5y, payload={row_id: r.id, expected, actual
        notifyOperator(high, "Audit chain mismatch at row=${r.id}")
        mismatch_count++
    prev_hash = r.record_hash
    cursor = r.id
while rows.length > 0

```

```

audit_logs(audit.chain.verify, 5y, payload={verify_date, run_id: runId, mismatch_count, tot

```

cron 設定: 0 17 * * *(UTC、毎日 17:00 UTC = 翌日 02:00 JST)、Cron 詳細は [DD08_Cron 実装.md](#) 参照。

所要時間目標: 1000 万行で 3600s 以内 (KV: monitoring: thresholds: chain-verify-duration で監視、超過時に分割検証へ切替)。

1.4.4 3.4 不一致検出時の通知ポリシー

検出件数	通知レベル	担当
1 件以上	high alert(運営者 inbox + メール強制送信)	当番運営者
累積 10 件以上 / 日	critical alert(オーナー + 全プロジェクト管理者 (project_users.role='admin' を 1 件でも保持するユーザー)、強制送信)	セキュリティ責任者

通知契機は [基本設計 / メッセージ一覧](#) §8 を正本とする。

1.4.5 3.5 監査ログ検索 API

```

/audit-logs:
get:
    summary: 監査ログ検索
    parameters:
        - in: query
          name: action
          schema: { type: string }
        - in: query
          name: from
          schema: { type: string, format: date-time }
        - in: query
          name: to
          schema: { type: string, format: date-time }
        - in: query
          name: cursor
          schema: { type: string }
        - in: query
          name: limit
          schema: { type: integer, default: 100, maximum: 200 }
    responses:
        "200":
            content:
                application/json:
                    schema:
                        type: object
                        properties:
                            items:
                                type: array
                                items:
                                    type: object
                                    nextCursor: { type: string, nullable: true }

```

検索自体も監査される (`audit.search`, `1y`, `{operatorId, filter, hitCount, ticketId}`)——APPI 30 条準拠。

性能目標: 100 万件以下で p95 $\leq$ 1000ms(KV: `monitoring:thresholds:audit-search-p95`)。インデックス: (`action`, `occurred_at`) 複合 + (`actor_id`, `occurred_at`) 複合。

1.4.6 3.6 監査エクスポート (HMAC 署名、D-17)

エクスポートは非同期ジョブ (`audit-export:<job_id>` KV TTL 24h) で実行する:

ステップ	処理
1	POST /audit-logs/exports(format: csv
2	Worker が D1 クエリ + R2 マルチパート PUT(10 万行/ファイル、5 並列上限)
3	全ファイル完了後 HMAC 署名 (HKDF info=audit-export、SHA-256) を .sig で生成
4	GET /audit-logs/exports/{job_id} で署名付き URL を取得 (R2 Pre-signed、TTL 24h)
5	エクスポート完了で audit.export(5y、{exportId, format, rows}) 記録

外部検証ツール: `admin/scripts/audit-export-verify.ts` で .sig + マスター 公開 OK の HKDF 派生 から HMAC 再計算 → 完全性確認。運営者は別保管した 情報で第三者検証可能。

HMAC 計算:

```
hmacKey = HKDF(masterKey, info="audit-export")
signature = HMAC-SHA256(
  key = hmacKey,
  message = sha256(file_content) || jobId || createdAt
)
```

性能目標: 10 万行/ファイルで $\leq$ 60s(KV: `monitoring:thresholds:audit-export-duration-p95`)。

1.4.7 3.7 3 区分保持 (D-08、付録 F)

retention_class	期間	物理削除タイミング	主な対象 action
general(1y)	365 日	RetentionPurgeWorker(日次 03:00 JST)	account.login / inbox.read / announcement.preview / pii_fp_report.create / audit.search
operator_high_priv(25y)	同上	同上	owner.* / operator.* / operator_approval.* / mfa.* / ai_parameter.update / master_key.rotate 等
billing(7y)	2555 日	同上	billing.* / stripe.event.processed / webhook.receive

物理削除前に R2 アーカイブ (`audit-archive/<retention_class>/<year>/<month>.tar.gz`) へ書出 → DELETE。完全な action_code × retention_class マップは [DD09_監査 action コード.md](#) を参照。

1.4.8 3.8 accounts_retired 永久保持

退会・物理削除されたオーナーは `accounts_retired` テーブルに移し、**法令対応のため永久保持**。audit_logs の target_id 参照を維持し、復元時のリンクを切らない。詳細は [基本設計 / セキュリティ設計](#) を参照。

1.4.9 3.9 構造化ログ (監査関連)

監査 INSERT 時に追加で構造化ログを出力する (Logpush / Logflare 等の集約に乗せる)。フォーマット:

```
{
  "level": "info",
  "worker": "AdminConsoleWorker",
  "request_id": "<cf-ray>",
  "trace_id": "01J...",
  "operator_id": "01J...",
  "action": "owner.suspend",
  "target_type": "owner",
  "target_id": "01J...",
```

```

    "retention_class": "operator_high_priv",
    "occurred_at": "2026-05-12T10:00:00Z",
    "ticket_id": "TKT-1234",
    "approval_id": "01J..."
  }

```

構造化ログの詳細スキーマは [DD12_運営者用ログ・テスト.md](#) §16 を参照。

1.4.10 3.10 主要エラーコード

エラー ID	HTTP	説明
E-OP-AUDIT-001	500	INTERNAL_CHAIN_INSERT_FAILED(リトライ後失敗)
E-OP-AUDIT-002	409	CHAIN_INSERT_CONFLICT(並行 INSERT 競合)
E-OP-AUDIT-003	400	EXPORT_FILTER_INVALID
E-OP-AUDIT-004	503	EXPORT_QUEUE_FULL(5 並列上限)

完全な E-OP-AUDIT-* 一覧は [基本設計 / エラー設計](#) を正本とする。

1.5 4. 関連設計

種別	参照先
要件	../01_要件定義/index.md
基本設計	../02_基本設計/index.md
セキュリティ設計(正本)	../02_基本設計/09_セキュリティ設計.md
テーブル設計(正本)	../02_基本設計/03_テーブル設計.md
関連 DD	DD08_Cron 実装.md / DD09_監査 action コード.md / DD12_運営者用ログ・テスト.md
運用設計	../04_運用設計/index.md
将来対応	../05_future/index.md

1.6 5. テスト観点

1.6.1 5.1 ユニットテスト

- HMAC-SHA256 + HKDF info=**audit-chain** 派生 の確定的計算
- **canonical_json** 正規化(キー順序ソート、Unicode NFC、数値表記)
- 単一 INSERT で **prev_hash + record_hash** チェーン整合
- 改ざんシミュレーション(1レコード書換 → 検証で不一致検出)
- HMAC 監査エクスポート署名計算 + 外部ツール検証

1.6.2 5.2 結合テスト (Miniflare)

- 1000 件 INSERT 後にチェーン全件検証 = 不一致 0
- 並行 INSERT で **prev_hash** 競合発生 → リトライ後成功
- 監査検索カーソルページング動作
- エクスポート(1000 行 × 3 ファイル)の HMAC 署名生成 + 検証

1.6.3 5.3 E2E テスト (Playwright)

テスト ID	シナリオ
e2e-scr096-001	検索 → 詳細 → エクスポート → HMAC 検証ツールでの完全性確認
e2e-audit-chain-001	チェーン検証バッチ実行 → 不一致なし
e2e-audit-chain-002	意図的に 1 レコード書換 → 検証バッチで検出 + 通知

1.6.4 5.4 負荷試験

シナリオ	構成	合格基準
(A1) 監査ログ大量検索	200 契約 × 5 年分 (約 550 万行) を 100 並列検索	p95 ☒ 1000ms
(A2) 監査ログ大量エクスポート	10 万行を 5 並列同時起動	全ジョブ完了 ☒ 90s、メモリ上限 128MB ヒット率 0%

1.6.5 5.5 受入条件マッピング

AC	検証手段
AC-038(運営者操作監査 + 通知)	E2E + 月次監査

1.7 6. 未確定事項・確認事項

確認事項 ID	確認内容	優先度	ステータス
-	v1.0 リリース時点で全項目確定済み	低	確認済